

Session Key Generator

Description:

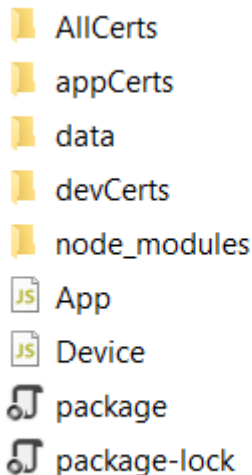
A nodeJS based utility to mimic lock and App.

Lock is simulated by device.js

App is simulated by app.js

Folder Structure:

Once installed at your end successfully, you should see the following folder structure:



AllCerts:

AllCerts folder contain **all** the certificates required. So it contains root, intermediate and device certificates. The broker side certificates are not required. We only use the device side of the certificate chain.

This folder is not being used currently. It's only there for reference and any cross checking that one may need to do.

appCerts:

All the certificates required by the app.

And app just requires the root certificate (root public certificate).

This is the emulation of the certificates that the mobile app will have

devCerts:

All the certificates required by the lock.

So it contains device and the intermediate certificate. These will be sent by the lock to the app and app will authenticate it against the root certificate.

This is the emulation of the certificates that the Lock will have

data:

data folder will be populated by device.js and app.js with `packet_dev.txt` and `packet_app.txt` respectively.

Since this PoC is for proving the encryption decryption mechanism and not the transport mechanism, the data sent from lock to app is stored in `packet_dev.txt` and data sent for app to lock is stored in `packet_app.txt`

device.js and app.js:

device.js is executed by calling `node .\device.js`

app.js is executed by calling `node .\app.js`

`device.js` runs in two stages:

First step it executes Step 2 (refer table **Secure Device On-boarding Concept** at bottom)

In this step it writes to `packet_dev.txt`

Then the user is expected to run `app.js` . (This executes)

Then in the second steps 3 and 4 from the below table.

Then device.js is allowed to continue further where it executes step 5 from the table.

Note

Make sure to run device.js and app.js in different terminals.

Procedure:

Step 1:

Terminal 1:

```
PS C:\Users\Admin> E:
PS E:\> cd .\proff\Clients\Welmade\SessionKeyGenerator\
PS E:\proff\Clients\Welmade\SessionKeyGenerator> node .\Device.js
```

[illegible]

Step 2:

Terminal 2:

```
PS E:\proff\Clients\Welmade\SessionKeyGenerator> node .\app.js
Starting the App.js application...
Step 1: Read the Package from the Device...
Packet received successfully.
{
  DEP1K: '04d024040df221626b8d6c944e55e7e4847210cf0d5e6ae15399e09a75eca56004ed4ee4b9557d5321ede91227c554b2847318e9aa924298ceebe7c7c77d24cf21',
  DS: 'MEYCIQC2kVAbfJYFUL+wDK4y7y0/Q5ngV4pNDDyKN8RFHQ0oHQIHAKMrUoaN0QJi0qUxop/SUqK8Aho7uD0H/mbs8iqxrQmA',
  DPC: '-----BEGIN CERTIFICATE-----\r\n' +
    'MIICvTCCAmOgAwIBAgIUefvQVHqUaFBSBqTw1go0cAdEihQwCgYIKoZIzj0EAwIw\r\n' +
    'gcIx CzA3BgNVBAYTAk1OMQswCQYDVQQIDAJNSDEMMaGA1UEBwwDUE5RMS0wKwYD\r\n' +
    'VQKDCRXZlxtYWRLIEExvY2tpbmV2cyBQcm12YXR1IEEx0ZC4xJTajBgNV\r\n' +
    'BAAsMHEVtYmVkbGVkIFN5c3R1bXMGRGV2ZWxvcG11bnQxEzARBgNVBAMMC1NtYXJ0\r\n' +
    'IEExvY2sxlTAzBgkqhkiG9w0BCQEWHmFkaXR5YS5heWFjaG10QGV1cm9wYWxvY2tz\r\n' +
    'LmNlVbTAeFw0yNjA2MTkwNDM5MzRaFw0zNjA2MTYwNDM5MzRaMIG1MQswCQYDVQ\r\n' +
    'EwJTTjElMAkGA1UECAwCTUgxDAAKBgNVBACMA1BOUTEtMCsGA1UECgwKv2Vs\r\n' +
    'ZSBsb2NraW5nIFN5c3R1bXMGRGV2ZWxvcG11bnQxEzARBgNVBAMMC1NtYXJ0\r\n' +
    'ZCBTeXN0ZW1zMRIwEAYDVQQDDA1MT0NLMTIzNDUxLTArBgkqhkiG9w0BCQEWHmFka\r\n' +
    'aXR5YS5heWFjaG10QGV1cm9wYWxvY2tzLmNlVbTBZMBMGBgYqGSM49AgEGCCGQSM49\r\n' +
    'AwEHAQIABkGA1UECAwCTUgxDAAKBgNVBACMA1BOUTEtMCsGA1UECgwKv2Vs\r\n' +
    'LyvoqaxD5v4vqmpTZjM2AFCI9nTCPvSjQjBAMB0GA1UdDgQNBBCFk+cFuedfNVWB\r\n' +
    'kqtvc0GZVhiYjYAFBgNVHSMEGDAWgBR2mbpaVr601a3qdSCL6j0QFbWg1DAKBggq\r\n' +
    'hkjOPQDAGNIADBFaIEAnnjHJJJaPxdhPnGhjoVUw/IlnmtXi1y6emJ0a7NHdUC\r\n' +
    'ICdCo1Y6cZg0FvD4/pjp7vEnMxv9KY0UDDCSW510GG8T\r\n' +
    '-----END CERTIFICATE-----\r\n',
  IDC: '-----BEGIN CERTIFICATE-----\r\n' +
    'MIIC8TCCApAgAwIBAgIUeMzUxhAE3DQYsMOAwN+kjb67D+z4wCgYIKoZIzj0EAwIw\r\n' +
    'gcUx CzA3BgNVBAYTAk1OMQswCQYDVQQIDAJNSDEMMaGA1UEBwwDUE5RMTAwLgYD\r\n' +
    'VQKDCdXZlxtYWRLIEExvY2tpbmV2cyBQcm12YXR1IEExpbW10ZWQxJTaj\r\n' +
    'BgNVBAAsMHEVtYmVkbGVkIFN5c3R1bXMGRGV2ZWxvcG11bnQxEzARBgNVBAMMCnNt\r\n' +
    'YXJ0IEExvY2sxlTAzBgkqhkiG9w0BCQEWHmFkaXR5YS5heWFjaG10QGV1cm9wYWxv\r\n' +
    'Y2tzLmNlVbTAeFw0yNjA2MTkwNTEzMzFaFw0zNjA2MTYwNTEzMzFaMIHCMQswCQYD\r\n' +
    'VQKGEwJTTjElMAkGA1UECAwCTUgxDAAKBgNVBACMA1BOUTEtMCsGA1UECgwKv2Vs\r\n' +
    'bWFKZSBMb2NraW5nIFN5c3R1bXMGRGV2ZWxvcG11bnQxEzARBgNVBAMMC1NtYXJ0\r\n' +
    'ZGR1ZCBTeXN0ZW1zMRIwEAYDVQQDDA1TbWYyZCBMb2NraW5nIFN5c3R1bXMGRGV2\r\n' +
    'KwYjKoZIhvcNAQkBFh5hZG10eWEuYX1hY2hpdEB1dXJvcGFsb2Nrcy5jb20wWTAT\r\n' +
    'BgqhkiG9w0BCQEWBmBwNCAATImN/MF13vHAu7xgKJu4Xuv2a4Zz1qWuxz\r\n' +
    'gOm24RMFZ13+TFNmzbPXUNtozo1AwXB03tjrTDqu1b5J3yID8zsjo2YwZDASBgNV\r\n' +
    'HRMBAf8ECDAGAQH/AgEAMA4GA1UdDwEB/wQEAwIBBjAdBgNVHQ4EFgQUdpm6Wla+\r\n' +
    'jont6g0gi+o6EBW1oJQwHwYDVR0jBBgwFoAU6QQ/Zh0oeTXQ6NL141dAHh84U0Sw\r\n' +
    'CgYIKoZIzj0EAwIDSAAwRQIgZjLo2WvZ64PE+SS6Xnh1WEsAK2QgtYv8YuoQOCC\r\n' +
    'QEwCIQDgpUQM/YH2T1syZNdwjHJBoIH2smInFwPe+/Wp44t7RQ==\r\n' +
    '-----END CERTIFICATE-----\r\n',
  DN: '4e62dfea6b2e37aba6e4150611911e15'
}
Step 2: verify certification chain...
Root certificate loaded successfully.
IDC valid: true
DPC valid: true
Step 3: Extract Device Public Key from Device Public Certificate...
```

```

Step 3: Extract Device Public Key from Device Public Certificate...
Extracted Device Public Key: -----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEJ2UvGQxCsDv6BVGARdFA55oqxWkb
/bQaz7tLw9wIUv+DQFgU5bvzzrkvK+iprEPm/i+qY9NmMzYAUij2dMI+9A==
-----END PUBLIC KEY-----

Step 4: Validate QR code and Device ID
QR code Device ID matches certificate Device ID.
Step 5: Verify Device Signature...
Step 6: Generate App's Ephemeral Key pair...
Generated App Ephemeral Public Key: 04177c50cbb6b12557686e337a2b73d13b178a3555b3beceb3701e2d292573a66c72da5d3cb05d7dc8af89aaacf8aa623fb03513c6aad3a8fb002b749f194ba5f
Generated App Ephemeral Private Key: 769bb15385b7252df5f7a5dd62d222602498fdFbdf07363ebd035c0e2f9bc295
Step 7: Generate shared secret...
Shared Secret: a809bc5892d395770015ed9994d5574bd4af28ac21a5e2945941dfaff2882065
Step 8: Generate shared symmetric AES Key...
sharedSymmetricAESKey:9e64eec4e7864780564aa7c306b7bc7fb5feb45d2e19a7147af82a31b17b6953
Step 9: Get SSID and Password from the user...
Hard coded right now: {"ssid":"MyHome","password":"abc123456"}
Step 10: Generate App Nonce...
Generated App Nonce: 9e59d1c52f916adb5d11a5bcbf4e2276
Step 11: Generate Cipher Text and Authentication Tag with Additional Authentic Data...
AAD buffer: {"DeviceID":"LOCK12345","DN":"4e62dfea6b2e37aba6e4150611911e15","Timestamp":"1782044501901"}
AAD buffer length:92
CipherText:3b318aee066f8ca677fa102b87e0a462eed9348817aee73de48505abcc1fe4b064a8fda46751c0f
Tag:1fc4b8bf7506aee797d48f44574ffe83
Step 12: Send package to device...
Packet written to packet_app.txt
Sending Package to App: {
  "AEP1K": "04177c50cbb6b12557686e337a2b73d13b178a3555b3beceb3701e2d292573a66c72da5d3cb05d7dc8af89aaacf8aa623fb03513c6aad3a8fb002b749f194ba5f",
  "IV": "9e59d1c52f916adb5d11a5bcbf4e2276",
  "AAD": "7b224465766963654944223a224c4f434b3132333435222c22444e223a223465363264666561366232653337616261366534313530363131393131653135222c2254696d657374616d70223a2231373832303434353031393031227d",
  "CT": "3b318aee066f8ca677fa102b87e0a462eed9348817aee73de48505abcc1fe4b064a8fda46751c0f",
  "TAG": "1fc4b8bf7506aee797d48f44574ffe83"
}
PS E:\proff\Clients\Welmade\SessionKeyGenerator>

```

Step 3:

Terminal 1:

Press **Enter** on Do you want to run the xtract part? :

```

Step 1: Read Package from App...
Packet received successfully.
{
  AEPIK: '04c3752f5110dc0d4f44125124726926609baecce53611bd2180e2884795bc1aec6914c5a4b12c899ab73ee42b5fdbe729014b8f6c840606b4a9e1c42e5fde68f8',
  IV: '0fe51fe1e4efbe387cb137037f0b59ea',
  AAD: '7b224465766963654944223a224c4f434b3132333435222c22444e223a2234653632646665613662326533376162613665343135303631373237356131222c2254696d657374616d70223a2231373832303434363939373939227d',
  CT: 'b90240cd9c3d5e95835928fe9532ad672db674ddbb42162c56a8ad97b74a360ef25f3b593d266437',
  TAG: 'f0a9641e232c44b06bd6979867eefeb6'
}
Step 1: Compute Shared Secret...
Device Ephemeral Pvt Key: 5b0fdd618cf9a4194cfc78612c813d1a49f3a6c8a46ee95a270afc13d9db6a2e
Computed Shared Secret: 6e2bcb03d7ee1f569da3bf1b8e5737c4a38ad30ed95c7038b4894c6062572bfe
Step 2: Generate Shared Secret Symmetric Key...
Computed Shared Secret Symmetric Key: ba1a45adae6e330cdc61fae5cfa0065fe91e6b77c304c716560f90cb198b0c92
Step 3: Verify Tag and Decrypt Credentials...
Authentication Tag Verified Successfully.
Decrypted Credentials JSON: {"ssid":"MyHome","password":"abc123456"}
SSID: MyHome
Password: abc123456

```

For trials with different ssid and passwords, you can edit the ssid and password in app.js :

```

37  var wifiCredentials =
38  {
39      ssid: 'MyHome',
40      password: 'abc123456'
41  };
42
43

```

Table:

Secure Device On-boarding Concept		
Implementation Logic		
Sr.no	Action	Description
1	Initiation	- User puts the device in AP mode using sequence of keys. - User scans the QR Code using mobile APP. - Mobile app Only gets the Device ID. - Using internal formulations mobile App derives the SSID and Password. - Now Mobile app sends connection request to the device.
2	Ephemeral Key Generation and Sharing (Device Side Implementation)	- The device generates the Device Ephemeral Private Key (DEPtK) and Device Ephemeral Public key(DEPIK) (<code> mbedtls_ecdh_gen_public()</code> ?) - Device generates a nonce (DN). A nonce is a 128 bit or 256 bit random number generated fresh for each new provisioning session. - A Device Hash (DH) is computed using Device ID, DEPIK and DN. - The DH is signed using Device Private Key (DPK) to create Device Signature(DS). - DH = SHA256(DeviceID DEPIK DN) - Now the device sends the following to the mobile: a. DEPIK. b. DS c. Device Public Certificate (DPC)

		d. Intermediate Device Certificate (IDC) e. DN
3	Authentication of the DEPIK at Mobile app	- Certificate chain verification: If certification chain is verified only then we move to the next steps. - Using Device Nonce, Device ID (the app has it because it has scanned the QR code), and DEPIK, the app creates local hash (LH). (LH = SHA256(DeviceID DEPIK DeviceNonce)) - Get the Device public key (DPIK) from DPC. - Verify the DS. (ECDSAVerify(DPIK,DS,LH)) - <i>If the verification succeeds it means that ***_DEPIK is authentic!***</i>
4	Shared Symmetric AES Key	- Now the app generates it's ephemeral key pair. (AEPtK, AEPIK) - ECDH is run on AEPtK + DEPIK to generate what can be called as a "shared secret". - This shared secret is run through KDF to generate Shared Symmetric AES Key (SSAK). Pass the Device Nonce and purpose of key generation to ensure uniqueness. (SSAK = HKDF-SHA256(SharedSecret, salt = DeviceNonce, info = "WiFiProvisioning")) - The app collects the Wifi Credentials from the user. - Generate a 12 byte nonce. We can call it as AppNonce (AN). Simply a 96 bit random number. - For more secure encryption adding Time stamp, Device ID and DeviceNonce is recommended. (This is called as Additional Authenticated Data) - This generates cipher text (CT) and Authentication Tag(Tag). - App creates a packet which consists of following: AEPIK : 65 bytes (P-256 uncompressed)

		AppNonce : 12 bytes (AES-GCM IV) AAD : variable Ciphertext : variable Tag : 16 bytes (AES-GCM Authentication Tag) This packet is now sent to the device.
5	Data Extraction at Device Level	- AES key generation at Device using AEPIK and DEPtK. For this run ECDH on DEPtK + AEPIK → Shared Secret, then KDF(Shared Secret) → SSAK → Verify Tag → Decrypt Wifi Credentials. 1. ECDH(DEPtK, AEPIK) → Shared Secret 2. HKDF(Shared Secret, DN, "WiFiProvisioning") → SSAK 3. Verify AES-GCM Authentication Tag 4. If verification succeeds: Decrypt and store WiFi credentials Else: Reject provisioning request
6	Note	For KDF use HKDF-SHA256 and for encryption use AES-256-GCM